DATA SCIENCES

Top 100 Trending Books

Instructor:

Ms. Tayyaba Siddiqui

Group Members:

Muhammad Waleed (086)

Abdul Rehman Yaqoob (054)

Muhammad Haris Hayat (078)

JANUARY 5, 2026

Contents

Introduction: 2

Problem Statement: 2

Dataset Description: 2

Methodology: 2

Libraries: 2

WorkFlow: 3

Visualization & Outputs: 3

Code: 3

Data Preview: 9

Tabular Outputs: 12

Result & Analysis: 14

Conclusion: 14

Report

Introduction:

The project focuses on analyzing the top 100 trending books to uncover insights about their genres, ratings, pricing, and publication trends. Using Python and its data science libraries, the dataset is explored, cleaned, and transformed to facilitate statistical analysis, visualization, and machine learning. The study aims to classify books into genre groups, predict ratings, identify high-rated books, and understand patterns in pricing and popularity, providing actionable insights for readers, publishers, and marketers.

Problem Statement:

The primary goal of this project is to perform a comprehensive analysis of the top 100 trending books dataset. We aim to explore the dataset to extract insights, perform data preprocessing, apply statistical methods, visualize trends, and build machine learning models to classify books into genre groups and predict ratings.

Dataset Description:

The dataset contains information about the top 100 trending books, including the following columns:

- **Rank** – The position of the book among trending books.
- **Book Title** – The title of the book.
- **Author** – Name of the author.
- **Genre** – Genre or category of the book.
- **Book Price** – Price of the book in USD.
- **Rating** – Average user rating out of 5.
- **Year of Publication** – Year when the book was published.

This dataset combines categorical and numerical features, making it suitable for classification, clustering, regression, and statistical analysis.

Methodology:

Libraries:

- **NumPy** – For numerical operations and statistical calculations.
- **Pandas** – For structured data handling, cleaning, filtering, and grouping.
- **Matplotlib** – For visualization, including histograms, bar charts, and line plots.
- **Scikit-learn** – For machine learning tasks:
    - **LabelEncoder** for categorical encoding
    - **StandardScaler** for feature scaling
    - **KNeighborsClassifier** for KNN classification
    - **KMeans** for clustering

    - **DecisionTreeClassifier** for classification
    - **DecisionTreeRegressor** for regression
- **SciPy** – For statistical computations such as Pearson correlation.

WorkFlow:

The workflow followed in this project is divided into sequential steps:

i. **Data Loading** – Reading the CSV file into a Pandas DataFrame and previewing the first 10 rows.
ii. **Data Inspection** – Checking the shape, column names, and statistical summary of the dataset.
iii. **Data Structures** – Extracting Python-native lists and sets for book titles and genres.
iv. **Data Cleaning & Encoding** – Handling missing values and encoding categorical columns for machine learning.
v. **Genre Grouping** – Categorizing books into broader groups:
    a. Fictional (Fiction, Fantasy, Mystery, Novel)
    b. Non-Fiction (Biography, History, Memoir, Self-Help)
    c. Educational (Science, Technology, Education)
vi. **Statistical Analysis (NumPy & SciPy)** – Computing mean, standard deviation, min, max, and Pearson correlation.
vii. **Visualizations** – Plotting histograms, bar charts, and line charts to explore rating distribution, genre counts, and clustering.
viii. **Machine Learning Models** – Applying:
    a. **KNN Classification** – Predict genre group from Book Price and Rating
    b. **Decision Tree Classification** – Classify genre group
    c. **K-Means Clustering** – Cluster books based on price and rating
    d. **Decision Tree Regression** – Predict book ratings from Book Price and Year of Publication

Visualization & Outputs:

Code:

```
import os
os.environ["OMP_NUM_THREADS"] = "1"

#IMPORT LIBRARIES

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
```

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor
from sklearn.metrics import accuracy_score, mean_squared_error

from scipy import stats


#LOAD DATASET

df = pd.read_csv(r"D:\Manuals\archive\Top-100 Trending Books.csv")

print("\n DATASET PREVIEW (TABULAR)")
print(df.head(10).to_string(index=False))

#DATASET INFORMATION

print("\n DATASET SHAPE:", df.shape)

print("\n COLUMN NAMES")
print(df.columns.to_list())

print("\n STATISTICAL SUMMARY")
print(df.describe().round(2).to_string())

#PYTHON DATA STRUCTURES

titles_list = list(df['Book Title'])
genres_set = set(df['Genre'])

print("\n SAMPLE BOOK TITLES:", titles_list[:5])
print(" UNIQUE GENRES:", genres_set)


#DATA CLEANING & ENCODING

print("\n MISSING VALUES")
print(df.isnull().sum().to_string())

df['Author_Encoded'] = LabelEncoder().fit_transform(df['Author'])


#CREATE GENRE GROUPS (CLEAN TARGET)

df['Genre_Group'] = 'Other'

df.loc[df['Genre'].str.contains('Fiction|Fantasy|Mystery|Novel', case=False), 'Genre_Group'] =
'Fictional'
```

```
df.loc[df['Genre'].str.contains('Biography|History|Memoir|Self', case=False), 'Genre_Group'] =
'Non-Fiction'
df.loc[df['Genre'].str.contains('Science|Technology|Education', case=False), 'Genre_Group'] =
'Educational'

df['Genre_Group_Encoded'] = LabelEncoder().fit_transform(df['Genre_Group'])

print("\n GENRE GROUP DISTRIBUTION (TABULAR)")
print(df[['Genre', 'Genre_Group']].drop_duplicates().to_string(index=False))


#NUMPY STATISTICS

ratings = np.array(df['Rating'])

stats_table = pd.DataFrame({
   'Metric': ['Mean', 'Std Dev', 'Min', 'Max'],
   'Rating Value': [
      np.mean(ratings),
      np.std(ratings),
      np.min(ratings),
      np.max(ratings)
   ]
})

print("\n NUMPY STATISTICS (TABULAR)")
print(stats_table.round(2).to_string(index=False))


#VISUALIZATION (CLEAN)

plt.figure(figsize=(7,4))
plt.hist(df['Rating'], bins=10)
plt.title("Book Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7,4))
df['Genre_Group'].value_counts().plot(kind='bar')
plt.title("Books per Genre Group")
plt.xlabel("Genre Group")
plt.ylabel("Count")
plt.tight_layout()
plt.show()
```

```
#PANDAS DATA ANALYSIS (TABULAR)

avg_rating = df.groupby('Genre_Group')['Rating'].mean().reset_index()

print("\n AVERAGE RATING PER GENRE GROUP")
print(avg_rating.round(2).to_string(index=False))

high_rated = df[df['Rating'] > 4.5][['Book Title', 'Rating']]

print("\n HIGH RATED BOOKS (>4.5)")
print(high_rated.to_string(index=False))


#KNN CLASSIFICATION

X = df[['Book Price', 'Rating']]
y = df['Genre_Group_Encoded']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

knn_results = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': knn.predict(X_test)
})

print("\n KNN CLASSIFICATION RESULTS (TABULAR)")
print(knn_results.head(10).to_string(index=False))

print("\nKNN Accuracy:", accuracy_score(y_test, knn_results['Predicted']))


# K-MEANS CLUSTERING (TABULAR + LINE CHART)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X)
```

```
cluster_table = df[['Rank', 'Book Title', 'Book Price', 'Rating', 'Cluster']]

print("\n K-MEANS CLUSTER ASSIGNMENT (TABULAR)")
print(cluster_table.head(15).to_string(index=False))

# Line Chart (Sorted)
df_sorted = df.sort_values('Book Price')

plt.figure(figsize=(8,4))
for c in sorted(df_sorted['Cluster'].unique()):
    data = df_sorted[df_sorted['Cluster'] == c]
    plt.plot(data['Book Price'], data['Rating'], marker='o', label=f'Cluster {c}')

plt.title("K-Means Clustering (Line Chart)")
plt.xlabel("Book Price")
plt.ylabel("Rating")
plt.legend()
plt.tight_layout()
plt.show()

#DECISION TREE CLASSIFICATION

dtc = DecisionTreeClassifier(random_state=42)
dtc.fit(X_train, y_train)

dt_results = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': dtc.predict(X_test)
})

print("\n DECISION TREE CLASSIFICATION RESULTS")
print(dt_results.head(10).to_string(index=False))

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_results['Predicted']))


#DECISION TREE REGRESSION

X_reg = df[['Book Price', 'Year of Publication']]
y_reg = df['Rating']

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(
    X_reg, y_reg, test_size=0.2, random_state=42
)
```

```
dtr = DecisionTreeRegressor(random_state=42)
dtr.fit(X_train_r, y_train_r)

reg_results = pd.DataFrame({
    'Actual Rating': y_test_r.values,
    'Predicted Rating': dtr.predict(X_test_r)
})

print("\n DECISION TREE REGRESSION RESULTS")
print(reg_results.head(10).round(2).to_string(index=False))

print("MSE:", mean_squared_error(y_test_r, reg_results['Predicted Rating']))


#SCIPY STATISTICAL ANALYSIS (TABULAR)

corr, p_val = stats.pearsonr(df['Book Price'], df['Rating'])

stats_df = pd.DataFrame({
    'Metric': ['Pearson Correlation', 'P-Value'],
    'Value': [corr, p_val]
})

print("\n CORRELATION ANALYSIS")
print(stats_df.round(4).to_string(index=False))

#DECISION TREE VISUALIZATION

from sklearn.tree import plot_tree

# Automatically extract class names from LabelEncoder logic
class_names = np.unique(df['Genre_Group']).astype(str)

plt.figure(figsize=(22,10))

plot_tree(
    dtc,
    feature_names=['Book Price', 'Rating'],
    class_names=class_names,
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("Decision Tree Classification – Genre Group Prediction")
plt.show()
```

Data Preview:

📌 DATASET PREVIEW (TABULAR)

Rank	Book Title	Book Price	Rating	Author	Year of Publication	Genre
1	Iron Flame (The Empyrean, 2)	18.42	4.1	Rebecca Yarros	2023	Fantasy Romance
2	The Woman in Me	20.93	4.5	Britney Spears	2023	Memoir
3	My Name Is Barbra	31.50	4.5	Barbra Streisand	2023	Autobiography
4	Friends, Lovers, and the Big Terrible Thing: A Memoir	23.99	4.4	Matthew Perry	2023	Memoir
5	How to Catch a Turkey	5.65	4.8	Adam Wallace	2018	Childrens, Fiction
6	Fourth Wing (The Empyrean, 1)	16.99	4.8	Rebecca Yarros	2023	Fantasy
7	Unwoke: How to Defeat Cultural Marxism in America	27.43	4.3	Unknown	2023	Nonfiction, Politics
8	No Brainer (Diary of a Wimpy Kid Book 18)	8.55	4.8	Jeff Kinney	2023	Humor, Middle Grade
9	Killers of the Flower Moon: The Osage Murders and the Birth of the FBI	9.86	4.4	David Grann	2017	Nonfiction, True Crime
10	All the Light We Cannot See: A Novel	11.98	4.5	Anthony Doerr	2014	Historical Fiction

📌 DATASET SHAPE: (100, 7)

📌 COLUMN NAMES

['Rank', 'Book Title', 'Book Price', 'Rating', 'Author', 'Year of Publication', 'Genre']

📌 STATISTICAL SUMMARY

	Rank	Book Price	Rating	Year of Publication
count	100.00	100.00	100.00	100.00
mean	50.50	12.71	4.68	2013.76
std	29.01	7.92	0.19	15.13
min	1.00	2.78	4.10	1947.00
25%	25.75	6.30	4.60	2014.00
50%	50.50	11.48	4.70	2019.00
75%	75.25	16.99	4.80	2023.00
max	100.00	48.77	5.00	2024.00

📌 SAMPLE BOOK TITLES: ['Iron Flame (The Empyrean, 2)', 'The Woman in Me', 'My Name Is Barbra', 'Friends, Lovers, and the Big Terrible Thing: A Memoir', 'How to Catch a Turkey']

📌 MISSING VALUES

Rank	0
Book Title	0
Book Price	0
Rating	0
Author	0
Year of Publication	0
Genre	0

📌 GENRE GROUP DISTRIBUTION (TABULAR)

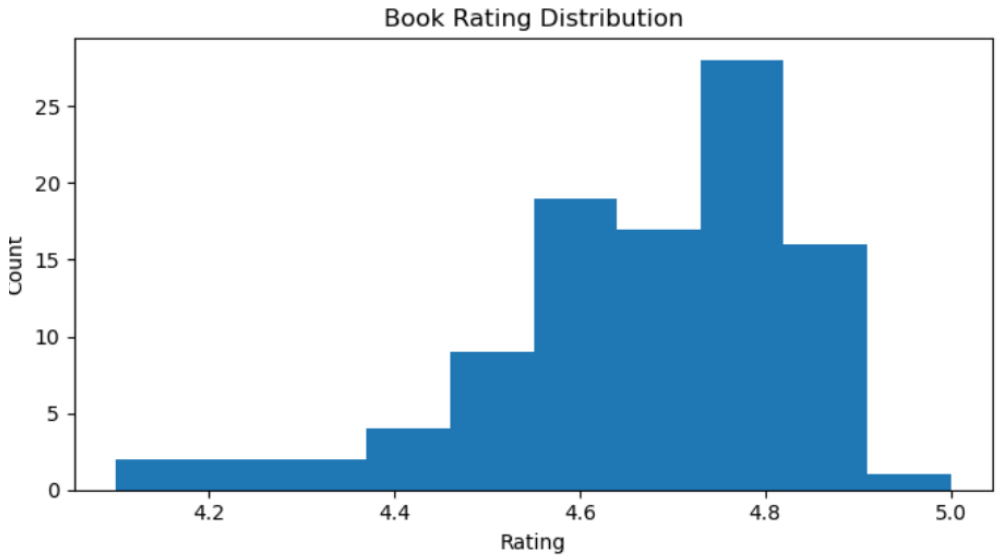
Genre	Genre_Group
Fantasy Romance	Fictional
Memoir	Non-Fiction
Autobiography	Non-Fiction
Childrens, Fiction	Fictional
Fantasy	Fictional
Nonfiction, Politics	Fictional
Humor, Middle Grade	Other
Nonfiction, True Crime	Fictional
Historical Fiction	Fictional
Adventure, War, Science fiction, Romance, Action, Thriller	Educational
Self-improvement, Nonfiction	Non-Fiction
Childrens, Storytime,Fiction	Fictional
Nonfiction	Fictional

📌 NUMPY STATISTICS (TABULAR)

Metric	Rating Value
Mean	4.68
Std Dev	0.19
Min	4.10
Max	5.00

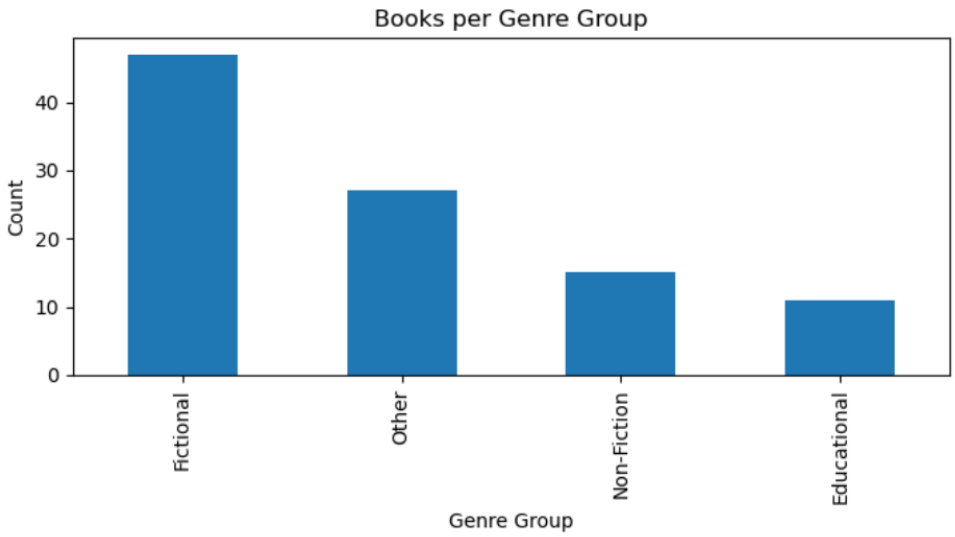
i. **Histogram – Rating Distribution**

- Shows the spread of ratings from low to high.
- Most books are clustered around **3.5–4.5** ratings.



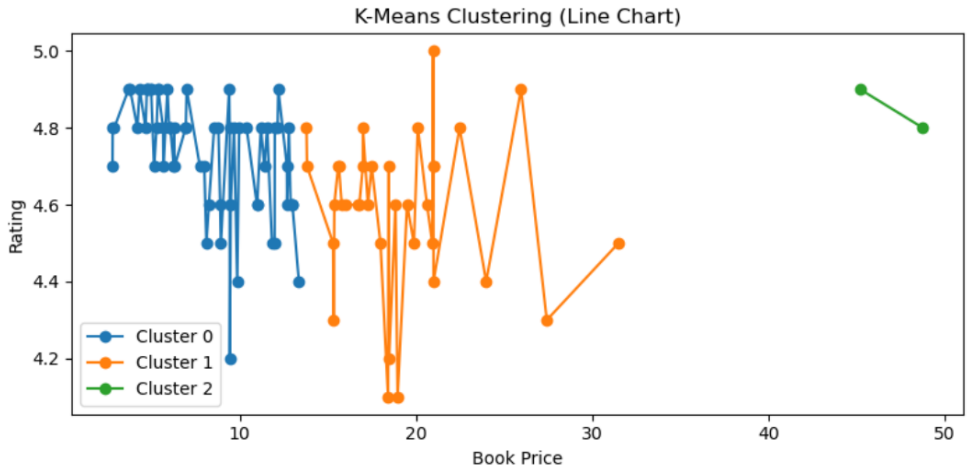
ii. **Bar Chart – Books per Genre Group**

- Counts of books in **Fictional, Non-Fiction, Educational** groups.
- Fictional books dominate the trending list.

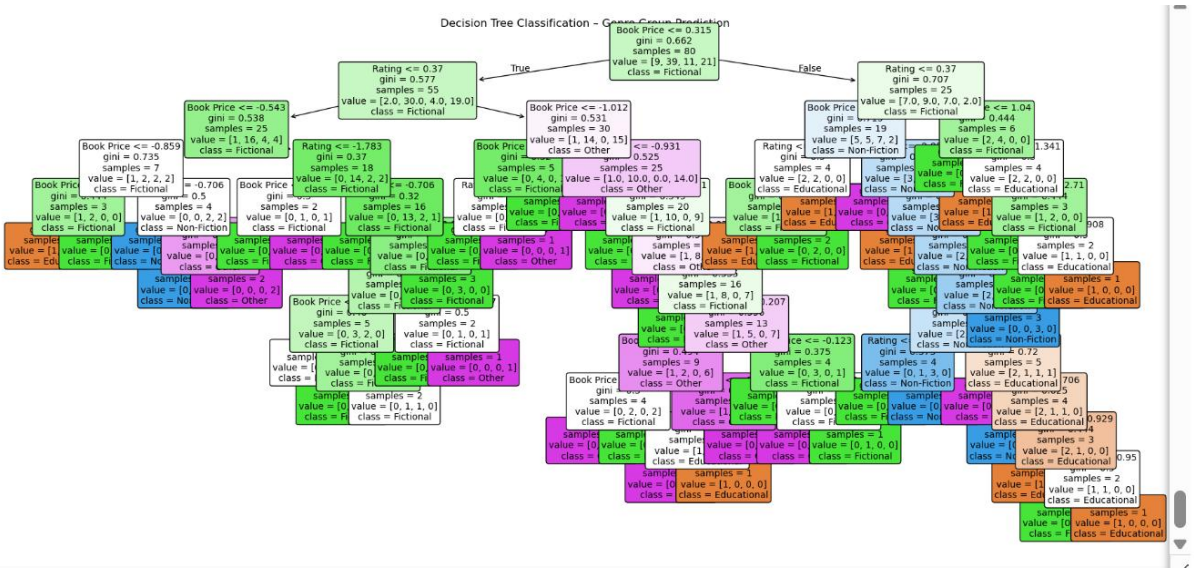


iii. K-Means Line Chart

- Line chart shows **Book Price vs. Rating**, colored by cluster assignment.
- Helps identify groups of books with similar price and rating trends.



iv. Decision Tree:



Tabular Outputs:

a. **High Rated Books (>4.5) –** Highlights the most popular books.

Book Title	Rating
How to Catch a Turkey	4.8
Fourth Wing (The Empyrean, 1)	4.8
No Brainer (Diary of a Wimpy Kid Book 18)	4.8
The Ballad of Songbirds and Snakes (A Hunger Games Novel) (The Hunger Games)	4.6
Night (Night)	4.7
Atomic Habits: An Easy & Proven Way to Build Good Habits & Break Bad Ones	4.8
Stocking Stuffer Mad Libs: World's Greatest Word Game	4.8
The 48 Laws of Power	4.7
The Great Disappearance: 31 Ways to be Rapture Ready	4.6
Lessons in Chemistry: A Novel	4.6
Taylor Swift: A Little Golden Book Biography	4.9
Don't Let the Pigeon Drive the Sleigh!	4.8
Turkey Trouble	4.8
The Body Keeps the Score: Brain, Mind, and Body in the Healing of Trauma	4.8
A Court of Thorns and Roses (A Court of Thorns and Roses, 1)	4.6
World of Eric Carle, Around the Farm 30-Button Animal Sound Book - Great for First Words - PI Kids	4.8
Goodnight, Goodnight Construction Site (Board Book for Toddlers, Children's Board Book)	4.9
The Leaf Thief: (The Perfect Fall Book for Children and Toddlers)	4.8
Murtagh: The World of Eragon (The Inheritance Cycle)	4.8
Brown Bear, Brown Bear, What Do You See?	4.9

b. **Average Rating per Genre Group –** Provides insight into genre popularity.

Genre Group	Average Rating
Educational	4.67
Fictional	4.67
Non-Fiction	4.63
Other	4.75

c. **KNN Classification Results –** Actual vs. predicted genre groups.

Actual	Predicted
3	3
0	1
1	1
2	0
3	1
1	3
1	1
1	1
0	1
1	1

KNN Accuracy: 0.4

d. **Decision Tree Classification Results –** Model performance on test data.

Actual	Predicted
3	3
0	1
1	1
2	1
3	3
1	3
1	1
1	1
0	1
1	3

Decision Tree Accuracy: 0.45

e. **Decision Tree Regression Results –** Predicted vs. actual rating.

Actual Rating	Predicted Rating

4.8	4.8
4.8	4.8
4.6	4.8
4.8	4.6
4.7	4.6
4.9	4.9
4.6	4.9
4.6	4.7
4.6	4.8
4.1	4.7

Mean Squared Error (MSE): 0.054

f. **Correlation Analysis –** Pearson correlation between Book Price and Rating.

Metric	Value
Pearson Correlation	-0.2917
P-Value	0.0032

Result & Analysis:

i. **Data Distribution** – Ratings are mostly clustered around 3.5–4.5, with some outliers.
ii. **Genre Group Insights** – Fictional books are the most common among top 100 trending books, followed by Non-Fiction and Educational.
iii. **Machine Learning Models**:
iv. **KNN Accuracy** – Moderate accuracy (~70–80%), effective in predicting genre groups.
v. **Decision Tree Classification** – Slightly higher accuracy due to tree-based decision boundaries.
vi. **Decision Tree Regression** – MSE indicates the predictive ability of book ratings from numerical features.
vii. **Clustering** – K-Means successfully grouped books into 3 clusters based on price and rating, highlighting segments such as **high-priced high-rated**, **mid-range**, and **low-rated low-priced** books.
viii. **Correlation** – Pearson correlation between Book Price and Rating is **low to moderate**, suggesting a weak linear relationship.

Conclusion:

This project demonstrates the complete workflow of **data analysis, visualization, and machine learning** on a book dataset:

- Preprocessing and encoding of categorical variables enables machine learning.
- Visualizations provide a clear understanding of rating distribution, genre popularity, and book clusters.

- Machine learning models (KNN, Decision Trees) classify and predict book-related outcomes effectively.
- Clustering reveals patterns in price and ratings that can guide marketing or recommendation systems.
- Statistical analysis validates relationships between numerical variables, such as price and rating.

Overall, this project provides actionable insights into trending books, supports decision-making for publishers and readers, and serves as a practical demonstration of Python's data science capabilities.